Copado Agentia Headless Virtual Hackathon

Agent Passport

A plugin for the Copado Agentia CLI that gives every Agentforce agent a passport: proof that it is complete, that it behaves as its user story says, and that nothing changed since. Copado will not move the story from Dev to QA without it.



Three checks

Before a story leaves Dev, Passport checks what the agent needs, how it behaves, and whether anything changed after testing.

Inside Copado's flow

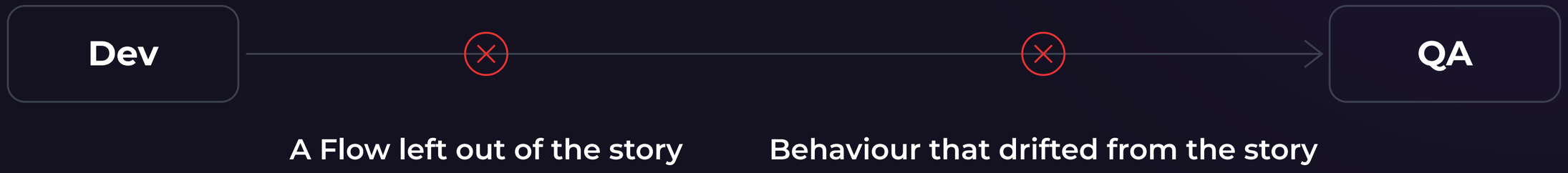
Agentia's own quality gate runs it on every `work test` and `work submit`. Nothing new to learn.

Proven live

On a real Copado story it blocked a broken agent, then stamped the fixed one and let it through.

The problem

Copado can promote an agent that is broken



An agent is many pieces, and its behaviour comes from a language model. A successful deploy proves the files arrived. It does not prove the agent works.

Pieces left behind

An agent calls Flows, Apex and prompts. If the story forgets one, the agent breaks at run time in QA.

Behaviour drifts

A small prompt edit can change routing or skip an action. No code check or Apex test notices.

Found too late

Today it shows up in QA or with customers, after the release moved on, when a fix costs the most.

The idea

Passport control between environments

Like a border, an agent may only cross from Dev to QA with a valid passport. The checkpoint is Agentia's local quality gate, and it asks three questions.



Packed?

`scan` maps every dependency. `travel` confirms each one is in the story or already in QA.

Behaves?

Each acceptance criterion becomes a test conversation, run 3 times in the Dev org.

Unchanged?

A stamp per environment, committed with the story, holds a fingerprint of every file.

Turn it on

One command plugs into Agentia

Run it once per repository and commit the files it writes. From then on Copado's own commands check the passport, and coding agents know the workflow.

```
> agentia passport setup --env Passport-Dev --travel-to Passport-QA
```

What it writes

A marked block in `.agentia_quality_gates.sh` and `.cmd` that runs `passport verify`, plus the agent-passport skill.

When it runs

`work test` and `work submit` run the gate first. A failing gate stops submit before any Git push or Copado call.

Safe to add

Existing gates like Apex tests and Code Analyzer keep running. `setup --remove` restores the scripts byte for byte.

What the agent needs

Every dependency, checked against QA

Scan reads the agent's source. Travel checks every piece against the QA branch it finds in the Copado pipeline.



Scan reads Agent Script or classic metadata down to Flows, Apex, prompts and the agent user, and flags broken references.

Travel marks each piece in story, on target or gap, using the target branch from the Copado pipeline.

2 deployment steps cover what metadata can't carry: the agent user, and publishing and activating the agent in QA.

A coding agent, through MCP

The same workflow, inside a coding agent

A developer can ask Claude Code to get a story's agent ready to promote. The skill tells it what to do in what order, and Agentia's MCP server lets it act in Copado.

Claude Code
The developer asks in plain English.

→

agent-passport skill
Teaches the steps and the rules.

→

Agentia MCP server
222 Copado tools, from `agentia mcp start`.

→

Copado story
Deployment steps, read and added live.

Rules the skill sets

Fix the agent, never weaken a test. Never edit evidence. Never use `--skip-local-tests`. The coding agent runs `sf`; the plugin only reads results.

What MCP adds

Passport proposes the deployment steps. The coding agent checks the story with `agentia_work_deployment_step_list` and adds missing ones with `..._create`.

Test the agent's behaviour

Acceptance criteria become tests

`goldens init` reads the story's acceptance criteria through Agentia and drafts one test conversation each. Every conversation runs 3 times, because a language model can route differently each time.

Drafted from the story. A criterion becomes an expectation only when it names a real subagent or action. Anything else is marked for review.

Run by you or your coding agent. `goldens plan` prints the exact `sf agent preview` commands, with live actions.

Checked turn by turn. Route, actions run or not run, reply text, and unfilled merge fields.

Story US-0000025 · Dev org · 3 runs each

AC-1	0 / 3	Refund request: escalate_case received the literal text {!state.issue_type}
AC-2	0 / 3	Damaged order: expected create_case, but it never ran

REFUSED All 3 runs must pass

Copado says no

Submit stops before QA

`passport verify` is the gate. It exits 1 unless every check passes, and Agentia stops.

```
> agentia cicd work test
scan    pass  no scan errors
stamp   FAIL  Passport-Dev stamp was refused: AC-1 (0%); AC-2 (0%)
travel  FAIL  will not arrive in Passport-QA: Flow:EscalateCase
PASSPORT NOT VALID for Passport-Dev.
Local quality gates failed (exit 1)
```

Scan

No errors: no broken references, missing Flows or hardcoded org usernames.

Stamp

A stamp exists for this environment, it was granted, and its fingerprint still matches.

Travel

Every dependency is in the story or already on the target branch.

The report

One page for reviewers

`passport report` writes one self-contained HTML file, committed with the story.

Verdict first: valid or not, with a stamp for each environment and the reason.

Every conversation: each run's transcript, route, actions and failed checks.

Travel: what will not reach QA, and the deployment steps it needs.

Evidence: the dependency map, fingerprints and hashes behind each stamp.



Fix and pass

Fix the agent, never the tests

The test conversations stayed exactly the same. Only the agent and the story changed.

AC-1 · routes to resolution

0% → 100%

AC-2 · creates a real case

0% → 100%

3 of 3 live runs in the Dev org. **STAMPED**

Create the case first

AC-2 failed because the agent talked about a case but never made one. It now runs create_case first.

Pass real values

AC-1 failed on an unfilled merge field. Escalation now gets the issue description and the case number.

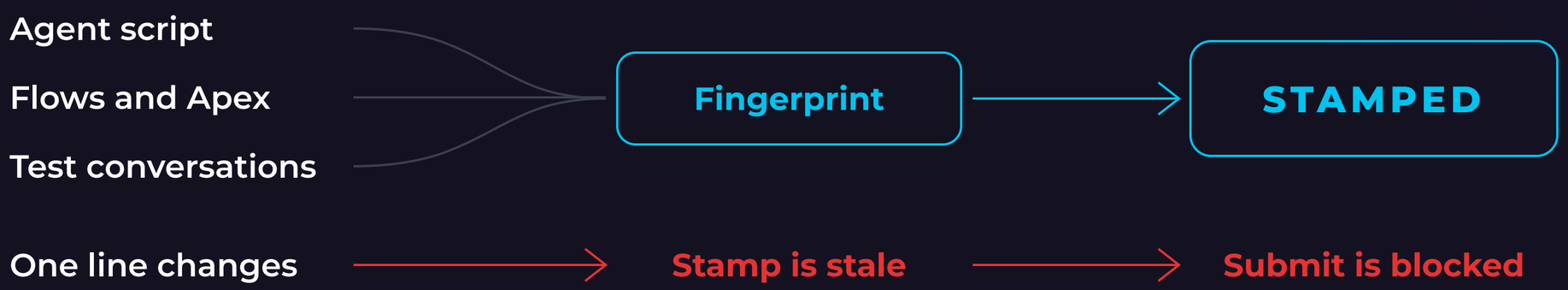
Commit the Flow

EscalateCase was on disk but not in the story. Committed, so travel passes.

Drift

The stamp is tied to the exact files

A granted stamp records a sha256 fingerprint of every file in the agent's dependency graph, plus a hash of the test conversations.



What counts

The agent script, every Flow, Apex class and prompt it uses, and the test file.

What happens

`verify` recomputes it. Any difference makes the stamp stale, and submit stops.

Why

Proof must match what ships. Re-run the tests and stamp again.

Agent Passport

Safer agent releases, with proof on every story

	Without Passport	With Passport
A missing Flow	Found in QA, after the deploy	Caught in Dev by travel, before submit
Behaviour changes	Found by testers or customers	Criteria run 3 times; stamp refused
Edits after testing	Invisible	Fingerprint makes the stamp stale
Reviewer's view	A code diff	One report with the real conversations

Built on Agentia

9 plugin commands on the host API, the quality gate, an agent skill and MCP.

Tested

189 automated tests on Windows and Linux, against the real Agentia CLI.

Proven live

Blocked, fixed, stamped and submitted on a real Copado story.

How it fits into Copado

From story to promotion

Nine steps, read left to right. The developer's day stays the same; Passport adds the proof.

Set story `work set`
Pick the Copado story. Passport reads its acceptance criteria.

Scan `passport scan`
Map every dependency and flag broken references.

Travel `passport travel`
Check each piece reaches QA and propose deployment steps.

Deploy steps `Agentia MCP`
The coding agent adds the agent user and publish steps.

Test `sf agent preview`
Criteria run as conversations in the Dev org, 3 times each.

Stamp `passport stamp`
Score the traces. Grant or refuse, with a fingerprint.

Submit `work submit`
The developer submits as usual. Nothing new to learn.

Gate `passport verify`
Not valid stops submit, with the reason. Valid carries on.

Copado `validate, promote`
Promotion as usual. The passport travels in Git.

Built and tested

Built to be trusted

189 tests

162 unit and 27 integration tests on Windows and Linux, including the real `agentia cicd work test`.

Real data

Salesforce's open-source sample agents, and real agent preview sessions from a Developer Edition org.

Clear boundaries

Copado data only through the host API. No credentials, and the plugin never touches an org.

Design choices

Evidence in Git. Stamps, baselines and tests are committed and reviewed like code.

No guessing. Test drafts never claim more than the story says.

Clear trace errors. An empty trace is named, with the preview command that fixes it.

Scriptable. Every command has `--json`, for pipelines and coding agents.

Thank you

Safer agent releases, with proof attached to every story.
Built on the Copado Agentia CLI.



Read more: the README and ARCHITECTURE files in the linked repository.